Stage 2 Implementation Report — DRAFT

November 8, 2026 (Toronto)

Contents

Stage 2 Implementation Report — DRAFT **1**

1. Project overview and implementation status . . . 1
2. Feature-to-implementation mapping . . . 2
3. Implemented design patterns . . . 6
4. Changes and rationale . . . 7
5. AI tools and collaboration log . . . 9
6. Development and verification summary . . . 10
7. Demonstration index . . . 11

Appendix A. Source-file index . . . 12

Appendix B. Reproduction and desktop verification . . . 12

Stage 2 Implementation Report — DRAFT

AI Job Search and Career Agent — EECS3311

Prepared for the repository owner. Personal identifiers omitted from this public working draft.

Implementation checkpoint: November 8, 2026 (Toronto). Submission due November 10, 2026.

Not ready for submission: live feature checks, all feature demonstration links/timestamps, actual AI usage/cost figures and final human review remain pending. The user reported successful live Groq connectivity verification on November 8. Five meaningful Period 3 code commits were pushed to main on November 7; the code/weekly-upload checkpoint is complete. The final report and videos must still be completed before submission.

1. Project overview and implementation status

The application serves students, new graduates and career changers who need a single workspace for profiles, resumes, job comparison, application tracking, interview practice, research and career planning. Both Swing GUI and CLI route through AgentController and CareerFacade to the same local records. Source code now contains workflows for all 17 original Stage 1 features; deterministic behavior and injected-provider workflows were verified with 61 passing tests, including a real desktop window and file picker. The user configured a private Free-plan key locally and reported a successful live Groq verification call on November 8; feature-level live checks remain pending.

ConfiguredLLMClient selects a concrete GroqAdapter by default (Groq-hosted openai/gpt-oss-20b) or an explicitly selected OpenAIAdapter (gpt-4.1-mini). Both use shared ChatCompletionAdapter HTTP translation behind provider-independent LLMClient. The Groq option was added after the user requested free access. Keys are provider-specific, and failures never trigger an automatic switch to a paid provider. The agent performs retrieval through ToolManager, task-specific generation, schema/evidence validation, review and persistence. It retrieves prior reviewed resume advice for career planning as durable advice memory. It is a bounded task agent, not an autonomous planner or unrestricted model-selected tool executor. Generated content never sends applications/messages or modifies the original resume automatically.

2. Feature-to-implementation mapping

All IDs below refer to the original Stage 1 feature with the same ID. “Implemented” describes connected source-code workflows, not a claim that live provider verification or feature videos have been completed. Every demonstration entry is pending recording and an accessible direct timestamped link.

F01 — Candidate Profile Management

- Stage 1 reference: F01. Status: implemented in code; live AI check pending where applicable.
- Mode: Deterministic.
- Participating classes/key methods: CareerFacade.updateProfile; CandidateProfile.updateProfile/updateSkills; Repository.save/loadProfile.
- User invocation: profile edit / Skills profile.
- Collaboration from input to output: Both views route candidate details through AgentController. The facade validates required name/education, normalizes skill sets and atomically saves the profile. Invalid entries leave the prior file intact.
- Design patterns: MVC, Facade.
- Demonstration video/timestamp: **pending**.

F02 — Resume Import

- Stage 1 reference: F02. Status: implemented in code; live AI check pending where applicable.
- Mode: Deterministic.
- Participating classes/key methods: CareerFacade.previewResume/previewResumeText/importResume; ResumeParser; DocumentParserAdapter; Resume; Repository.
- User invocation: resume import / resume paste / Resumes.
- Collaboration from input to output: The parser Adapter extracts PDF/DOCX text. The facade snapshots the source, calculates its checksum and presents a preview; explicit confirmation saves an immutable version. Unsupported, corrupt, blank and encrypted files fail without replacing existing versions; pasted text supports scanned resumes.
- Design patterns: Adapter, MVC, Facade.
- Demonstration video/timestamp: **pending**.

F03 — Resume Skill Extraction

- Stage 1 reference: F03. Status: implemented in code; live AI check pending where applicable.
- Mode: AI with deterministic validation.
- Participating classes/key methods: CareerAgent.extractSkills; ToolManager.readProfile/readResume; SkillProposal; CareerFacade.approveSkills; AgentController.
- User invocation: resume extract-skills / Resumes.
- Collaboration from input to output: The task retrieves a saved resume and profile, calls LLMClient, validates source excerpts and full skill-token boundaries, removes duplicates and exposes numbered proposals. Selected skills merge with the current profile only after candidate approval.
- Design patterns: Adapter, Facade, MVC.
- Demonstration video/timestamp: **pending**.

F04 — Job Import

- Stage 1 reference: F04. Status: implemented in code; live AI check pending where applicable.
- Mode: Deterministic.
- Participating classes/key methods: CareerFacade.importJob; Job.updateDetails; Repository.findJobByTitleAnd
- User invocation: job import / Jobs & matching.
- Collaboration from input to output: The controller parses posting fields and an optional date. The facade creates a job or updates the matching title/company. Changing description text clears old requirements and evidence; changing unrelated metadata retains analysis.
- Design patterns: MVC, Facade.
- Demonstration video/timestamp: **pending**.

F05 — Job Description Analysis

- Stage 1 reference: F05. Status: implemented in code; live AI check pending where applicable.
- Mode: AI with deterministic validation.
- Participating classes/key methods: CareerAgent.analyzeJob; ToolManager.readJob; JobAnalysis; Requirement; CareerFacade.approveJobAnalysis; Job.updateReviewedAnalysis.
- User invocation: job analyze / job requirements / Jobs & matching.
- Collaboration from input to output: This task separately retrieves a job description and validates its requirement schema, category and supporting excerpt. Required/preferred skills, qualifications, experience and uncertain statements are reviewed by number. A source snapshot prevents approval after the description changes. Manual reviewed requirements remain available.
- Design patterns: Adapter, Facade, MVC.
- Demonstration video/timestamp: **pending**.

F06 — Skill Gap Identification

- Stage 1 reference: F06. Status: implemented in code; live AI check pending where applicable.
- Mode: Hybrid.
- Participating classes/key methods: ToolManager.compareSkills; CareerFacade.identifyGaps/explainGaps; CareerAgent.explainGaps; Assessment.
- User invocation: job gaps / job gaps explain.
- Collaboration from input to output: Approved skills are compared with reviewed requirements to produce matched, missing and uncertain lists. An optional task retrieves those facts and asks for an evidence-cited interpretation. An invalid or unavailable model result produces a warning alongside the deterministic result.
- Design patterns: Strategy context, Adapter, Facade, MVC.
- Demonstration video/timestamp: **pending**.

F07 — Job Matching

- Stage 1 reference: F07. Status: implemented in code; live AI check pending where applicable.
- Mode: Hybrid.
- Participating classes/key methods: MatchingStrategy; WeightedMatching; RequiredSkillMatching; ToolManager.calculateMatch/setMatchingStrategy; CareerFacade.matchJobs/explainMatches; CareerAgent.explainMatches.
- User invocation: job match / job match explain.
- Collaboration from input to output: A selected Strategy computes scores from approved skills, sorts descending and breaks ties by job ID. Weighted matching allocates 70% to required and 30% to preferred coverage when both exist; required matching uses required coverage. An optional task interprets the fixed ranking and gaps without controlling numeric results.
- Design patterns: Strategy, Adapter, Facade, MVC.
- Demonstration video/timestamp: **pending**.

F08 — Job Comparison

- Stage 1 reference: F08. Status: implemented in code; live AI check pending where applicable.
- Mode: Hybrid.
- Participating classes/key methods: CareerFacade.compareJobs; CareerAgent.summarizeComparison; ToolManager; Assessment.
- User invocation: job compare / Compare jobs.
- Collaboration from input to output: The facade validates two to four distinct selections, retrieves posting details and computes compatibility using the selected policy. It preserves unknown salary/location/deadline and unanalysed compatibility. A separate AI interpretation cites supplied comparison facts; failure preserves the table.
- Design patterns: Strategy, Adapter, Facade, MVC.
- Demonstration video/timestamp: **pending**.

F09 — Customized Resume Suggestions

- Stage 1 reference: F09. Status: implemented in code; live AI check pending where applicable.
- Mode: AI with review.
- Participating classes/key methods: CareerAgent.suggestResume; ToolManager.candidateEvidence/writingFin CareerDocument; CareerFacade.saveDocument; LocalRecords.
- User invocation: resume suggest / Writing assistant.
- Collaboration from input to output: The task retrieves the original resume, approved profile and reviewed job requirements. Each proposed suggestion needs a verbatim candidate excerpt. The candidate edits and approves a separate document; changed source records reject stale approval. Earlier reviewed advice is later retrieved as career-planning memory.
- Design patterns: Adapter, Facade, MVC.
- Demonstration video/timestamp: **pending**.

F10 — Cover Letter Drafting

- Stage 1 reference: F10. Status: implemented in code; live AI check pending where applicable.
- Mode: AI with review.
- Participating classes/key methods: CareerAgent.draftLetter; CareerDocument; CareerFacade.draftLetter/saveDocument/getDocument/listDocuments; LocalRecords.
- User invocation: letter draft / Writing assistant.
- Collaboration from input to output: The task validates a professional/warm/concise tone and retrieves candidate evidence and the selected posting. It validates two to six paragraphs and candidate quotations; the application supplies a generic hiring-team greeting and candidate signature. The editable letter is saved after review; provider failure preserves previously saved drafts.
- Design patterns: Adapter, Facade, MVC.
- Demonstration video/timestamp: **pending**.

F11 — Interview Question Generation

- Stage 1 reference: F11. Status: implemented in code; live AI check pending where applicable.
- Mode: AI with validation.
- Participating classes/key methods: CareerAgent.generateQuestions; QuestionSet/Question; InterviewService.generateQuestions; LocalRecords.
- User invocation: interview questions / Interview preparation.
- Collaboration from input to output: The task retrieves reviewed job requirements, requests an exact count of 1–20 questions and validates count, duplicate wording, technical/behavioural categories, rubrics and source quotations. With two or more questions both categories are required. Valid question sets are saved with their original job context.
- Design patterns: Adapter, Facade, MVC.
- Demonstration video/timestamp: **pending**.

F12 — Mock Interview

- Stage 1 reference: F12. Status: implemented in code; live AI check pending where applicable.
- Mode: Hybrid.
- Participating classes/key methods: InterviewSession.answer/pause/resume/finish; InterviewService.start/answer/transition; LocalRecords.
- User invocation: interview start / answer / skip / pause / resume / finish.
- Collaboration from input to output: A session reuses a saved question set and retains its original job/rubric snapshot. Each answer or explicit skip is saved. A paused session must resume before answering; finishing requires every question to be answered or skipped. State and next-question progress survive restart.
- Design patterns: Facade, MVC.
- Demonstration video/timestamp: **pending**.

F13 — Interview Answer Feedback

- Stage 1 reference: F13. Status: implemented in code; live AI check pending where applicable.
- Mode: AI with validation.
- Participating classes/key methods: CareerAgent.evaluateAnswers; AnswerFeedback; InterviewSession.attachFeedback; InterviewService.feedback.
- User invocation: interview feedback / Interview preparation.
- Collaboration from input to output: The task retrieves the session's submitted answers, original questions, rubrics and job context. It requires exactly one feedback entry per answered question, excludes skips, validates integer scores of 0–100, structured strengths/weaknesses/improvements and an exact answer excerpt. The complete response is validated before replacement of saved feedback.
- Design patterns: Adapter, Facade, MVC.
- Demonstration video/timestamp: **pending**.

F14 — Application Tracking

- Stage 1 reference: F14. Status: implemented in code; live AI check pending where applicable.
- Mode: Deterministic.
- Participating classes/key methods: CareerFacade.trackApplication/updateApplicationStatus; Application.updateStatus/restoreHistory; Repository.
- User invocation: application add / application update / Applications & deadlines.
- Collaboration from input to output: Tracking opens or creates the selected job's application and validates statuses, allowed transitions and nonfuture submission/saved dates in Toronto time. Status history persists. Duplicate applications reuse existing records; invalid transitions or dates are rejected.
- Design patterns: MVC, Facade.
- Demonstration video/timestamp: **pending**.

F15 — Deadline Tracking

- Stage 1 reference: F15. Status: implemented in code; live AI check pending where applicable.
- Mode: Deterministic.
- Participating classes/key methods: DeadlineManager.listDeadlines/checkDeadlines/subscribe/unsubscribe; DeadlineObserver.onDeadline; DeadlineReminder; GUI Timer; CLI.
- User invocation: deadlines list / deadlines check.
- Collaboration from input to output: The Observer subject uses an injectable Clock with Toronto default. It lists outstanding jobs as overdue, today, upcoming or unknown; submitted/closed applications are suppressed. Due dates within 0–7 days notify subscribed views once per local day/process. GUI checks every minute; CLI checks at each prompt. Views unsubscribe on close.
- Design patterns: Observer, MVC, Facade.
- Demonstration video/timestamp: **pending**.

F16 — Company Research Notes

- Stage 1 reference: F16. Status: implemented in code; live AI check pending where applicable.
- Mode: Hybrid.
- Participating classes/key methods: CompanyNotes; CareerFacade.saveCompanyNotes/getCompanyNotes; ToolManager.readNotes; CareerAgent.summarizeNotes; CareerDocument; LocalRecords.
- User invocation: company notes / company show / company summarize.
- Collaboration from input to output: User-entered research and optional HTTP(S) source URLs are stored by company. A separate task retrieves those notes, validates quoted evidence and citations against supplied URLs and returns an editable summary. It does not browse websites or verify the source contents. Changed notes invalidate stale summary approval; manual research persists despite AI failure.
- Design patterns: Adapter, Facade, MVC.
- Demonstration video/timestamp: **pending**.

F17 — Career Development Plan

- Stage 1 reference: F17. Status: implemented in code; live AI check pending where applicable.
- Mode: AI with deterministic constraints and review.
- Participating classes/key methods: ToolManager.planEvidence; CareerAgent.generateCareerPlan; CareerDocument; CareerFacade.generateCareerPlan/saveDocument; LocalRecords.
- User invocation: career plan / Career plan.
- Collaboration from input to output: The task retrieves profile, reviewed gaps for 1–4 selected jobs and up to five prior reviewed resume suggestions as advice memory. It validates a specific goal, weekly hours and cost budget; generated activities must reference known gaps, have positive study time and fit total time/cost limits. The candidate reviews a separate one-week plan; source changes invalidate stale approval.
- Design patterns: Adapter, Facade, MVC.
- Demonstration video/timestamp: **pending**.

3. Implemented design patterns

MVC

JobSearchGUI and JobSearchCLI are views. AgentController routes action/input maps and holds pending review snapshots; CareerFacade and domain classes own workflow rules; Repository/LocalRecords persist state. This prevents either UI from implementing independent validation/provider/persistence workflows. Key boundary: submitRequest(action,input). Native GUI requests run in SwingWorker while controls prevent overlapping writes.

Facade

CareerFacade coordinates CareerAgent, ToolManager, ResumeParser, DeadlineManager, InterviewService and Repository. Public feature methods expose meaningful operations such as approveJobAnalysis, saveDocument, startInterview and generateCareerPlan, rather than requiring a view to orchestrate subsystems. This matches the Stage 1 architectural role; explicit typed task values replace planned generic Request/Result objects.

Adapter

LLMClient is the target interface. GroqAdapter and OpenAIAdapter are concrete provider Adapters, sharing ChatCompletionAdapter for compatible HTTP translation, completion validation and error redaction. ConfiguredLLMClient reads the explicit provider choice locally for each request. The current default is Groq Free-plan access; optional OpenAI support is retained. ResumeParser is a second target with DocumentParserAdapter wrapping PDFBox/Apache POI via extractText(Path). Both adapter families count as one distinct pattern. Provider/parsing libraries do not enter UI workflows.

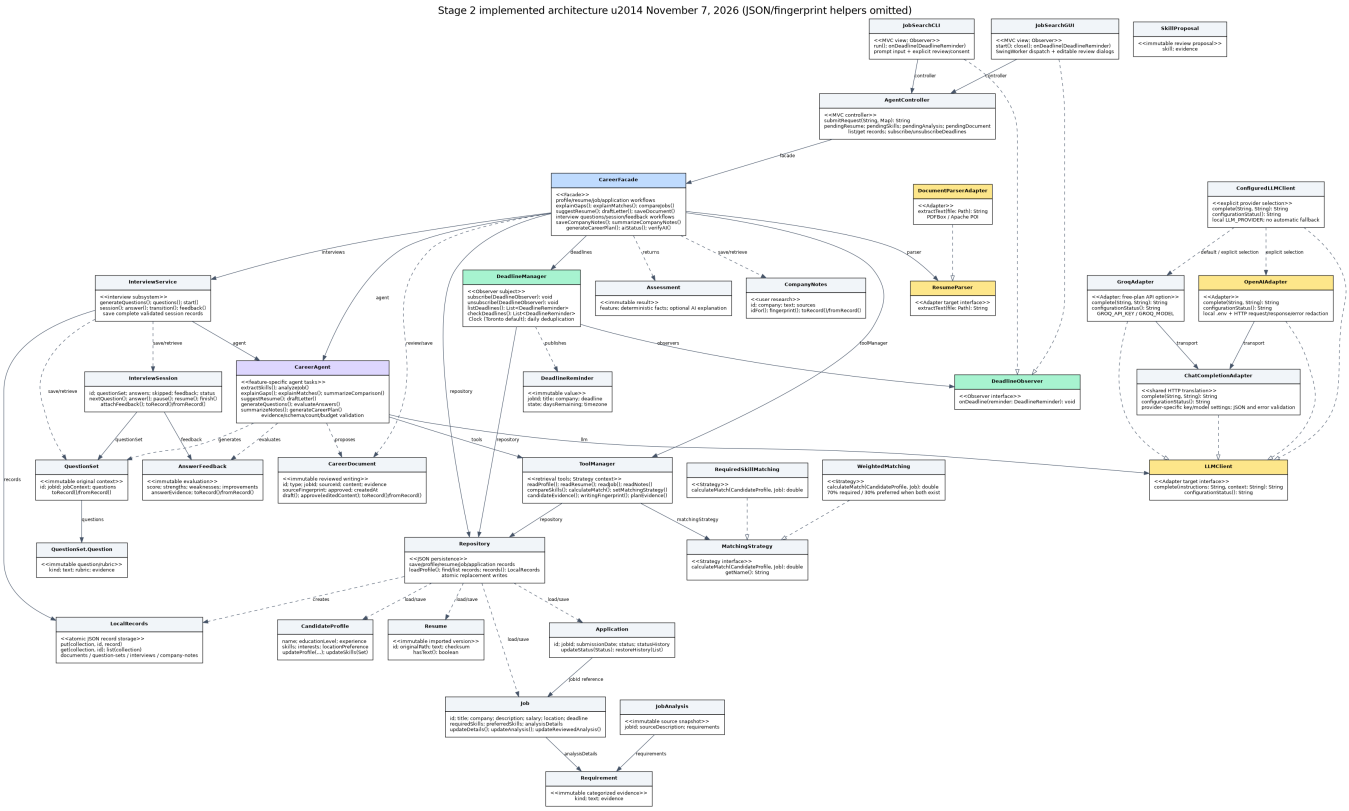
Strategy

MatchingStrategy defines calculateMatch(profile,job) and getName. WeightedMatching and RequiredSkillMatching are interchangeable policies. ToolManager is the context, and CareerFacade selects a policy and ranks results. Numeric matching stays deterministic even when optional model explanations fail. This matches the Stage 1 purpose.

Observer

DeadlineManager is the subject, maintaining subscribed DeadlineObserver instances. Views implement onDeadline(DeadlineReminder), register while active and unsubscribe on close. checkDeadlines retrieves current records and Clock-based Toronto dates; the subject does not import either UI. Injected Clock supports tests at local-day and seven-day boundaries. This refines the originally abstract notification signatures into concrete operations.

The implemented UML includes the November 7 writing, interview, research, planning and storage classes. Concrete Adapter/Observer operations and role labels are used; feature-specific sequences show retrieval, validation, failure and review paths. Original Stage 1 artifacts remain historical planned-design references.



The full-resolution class diagram, vector diagram, feature sequences and use cases accompany this report.

4. Changes and rationale

Generic executeTask/executeTool loop

- Original design: Generic executeTask/executeTool loop.
- Implemented change: Separate F03/F05/F06/F07/F08/F09/F10/F11/F13/F16/F17 methods with task-specific retrieval, schemas, evidence and failure branches.
- Reason: Professor requested feature-specific retrieval/validation; bounded calls make behavior testable.
- Impact: CareerAgent, ToolManager, CareerFacade; no unimplemented generic loop is claimed as complete functionality.

Resume/profile ownership and planned Request/Result transport types

- Original design: Resume/profile ownership and planned Request/Result transport types.
- Implemented change: Single local profile, independent resume/job/application records, action/input maps and immutable typed task results.
- Reason: Keep both views consistent while supporting durable single-candidate workflows.
- Impact: AgentController review state, Repository and new Assessment/JobAnalysis/SkillProposal/CareerDocu value types.

Loose job analysis output

- Original design: Loose job analysis output.
- Implemented change: Categorized reviewed requirements with verbatim evidence and source snapshot.
- Reason: Prevent ungrounded/stale requirements from affecting scoring.
- Impact: JobAnalysis, Requirement, Job.updateReviewedAnalysis; description changes clear old analysis.

Abstract parser/provider operations

- Original design: Abstract parser/provider operations.
- Implemented change: ResumeParser.extractText(Path): String; LLMClient.complete(String,String): String.
- Reason: Make concrete Adapter operations precise.
- Impact: DocumentParserAdapter wraps PDFBox/POI; OpenAIAdapter wraps chat-completions HTTP and local configuration.

Abstract deadline notification of an Application

- Original design: Abstract deadline notification of an Application.
- Implemented change: subscribe/unsubscribe/checkDeadlines/listDeadlines with onDeadline(DeadlineReminder), injectable Clock.
- Reason: Separate deadline rules from either UI and make timezone boundaries testable.
- Impact: Observer subject DeadlineManager; GUI Timer/CLI prompt checks; no external notification service.

Directly edited resume/letter output

- Original design: Directly edited resume/letter output.
- Implemented change: Reviewed CareerDocument records separate from original resumes, with source fingerprints and evidence.
- Reason: Preserve originals and require user review.
- Impact: F09/F10/F16/F17 share document review/persistence while retaining feature-specific generation and validation.

Generate questions for every new mock session

- Original design: Generate questions for every new mock session.
- Implemented change: F11 saves reusable QuestionSet records; F12 starts from a selected set and snapshots it.
- Reason: Avoid paying for regeneration and preserve the original evaluation rubric after job edits.
- Impact: InterviewService, InterviewSession; answers/skips/status/feedback are durable.

General career planning budget concept

- Original design: General career planning budget concept.
- Implemented change: Explicit one-week time and cost limits; validate actual generated totals.
- Reason: Reject over-time/over-cost recommendations rather than only suggesting review.
- Impact: F17 activity schema and decimal total checks; cost budget 0 requests free activities; prior reviewed advice is retrieved as memory.

R1/R2 relation identifiers

- Original design: R1/R2 relation identifiers.
- Implemented change: Role names and no redundant labels on realization edges; updated implemented-system diagrams.

- Reason: Professor's primary revision request.
- Impact: Refined Stage 1 export and separate current Stage 2 diagrams; preserve planned diagrams as historical references.

Direct JSON updates and unstable collection fingerprints

- Original design: Direct JSON updates and unstable collection fingerprints.
- Implemented change: Atomic replacement saves, canonical ordered source text, separate-JVM regression verification.
- Reason: Protect earlier records on save failure and keep approved documents editable after restart.
- Impact: Repository/LocalRecords, SourceFingerprint, ToolManager; bracket-containing/long array values round-trip.

Planned OpenAI-only integration → configurable free-provider default

- Original design: OpenAIAdapter with GPT-4.1 mini as the proposed runtime provider/model.
- Implemented change: GroqAdapter and ConfiguredLLMClient add Groq-hosted `openai/gpt-oss-20b` as the default; OpenAIAdapter remains an explicit optional selection. ChatCompletionAdapter shares compatible HTTP handling, with separate key/model namespaces and no automatic provider fallback.
- Reason: the user requested a free-provider option. Groq's documented Free plan and JSON-capable production model fit the existing task schemas. The Stage 2 instructions permit a cheap/free model and documented provider changes.
- Impact: all AI features use the same LLMClient boundary; per-feature retrieval/validation/review remains unchanged. Consent text is provider-neutral. The AI setup tab, CLI, safe example configuration, HTTP tests and model/provider report mapping were updated.
- Sources checked November 7, 2026: Groq Free-plan FAQ, rate limits, model, API compatibility. Availability and quotas are account/provider controlled; the user reported a successful live key check on November 8.

5. AI tools and collaboration log

Development tool: OpenAI Codex, GPT-6-based assistant; exact deployed model identifier is not exposed to this workspace. It assisted with implementation, code review, tests, fault diagnosis, design mapping and report drafting. The application runtime now defaults to Groq-hosted openai/gpt-oss-20b through GroqAdapter; optional OpenAI remains configurable through OpenAIAdapter. These runtime models are separate from the development-tool model.

Representative interactions:

- October 29 checkpoint: PDF/DOCX import through Adapter, preview/confirmation, versioned persistence and tests.
- November 1: user requested four meaningful Period 2 commits and supplied Stage 1/Stage 2 PDFs. Professor feedback required role labels and feature-specific retrieval/validation. Implemented F03, F05, F15 and shared GUI; 33 tests passed; pushed four commits November 1.
- November 7: implemented F06–F08, F09–F10, F11–F13 and F16–F17, then hardened cross-process source verification, persistence, identifiers/date validation and API setup. Five substantive code commits were pushed to main. A temporary workspace disconnect was recovered. A further tracker-history display regression was corrected; the desktop-enabled suite passed 61 checks after adding the free-provider adapter and its HTTP/configuration/quota tests.
- November 8: guided Windows JDK/Maven setup and private Groq configuration. The user supplied CLI output showing `ai status` selecting Groq and `ai verify` successfully receiving the expected JSON response. This establishes reported live connectivity; feature demonstrations and output review remain pending.

Human responsibilities: review and understand generated code, validate real provider access/output, record running feature demonstrations and review final submission. Exact total development-tool token use and monetary cost are **pending actual session usage/billing data**. Do not substitute guessed

figures or zero. Runtime API verification and demonstration costs are also pending actual provider usage/billing.

6. Development and verification summary

Weekly commits (Toronto local periods)

Only five distinct substantive development changes are selected for each completed period. Duplicated imports/merge copies and README-only updates are not used to inflate the counts.

Period 1 — October 20–26, 2026

- 8542dbe — Core domain models and JSON repository.
- 4ac8717 — F01 candidate profile through MVC and Facade.
- 0945542 — F04 job import and duplicate updates.
- 1509555 — F14 application tracking/history.
- c9cfb2d — Strategy matching and Period 1 verification.

Period 2 — October 27–November 2, 2026

- 80fbb8e — F02 PDF/DOCX resume import and verification.
- 5d74f5d — F03 reviewed resume skill extraction.
- 0c92787 — F05 reviewed job analysis and invalidation.
- df2882e — F15 deadline Observer.
- 12fc14a — Shared Swing GUI and refined UML/workflows.

Period 3 — November 3–9, 2026

- a73d1ac — Complete F06-F08 AI explanations and deterministic job comparison. Created November 07 at 16:48 Toronto.
- bbe8de8 — Implement F09-F10 editable evidence-backed resume and cover-letter drafts. Created November 07 at 16:52 Toronto.
- 7bd7015 — Implement F11-F13 interview questions, persistent sessions and rubric feedback. Created November 07 at 16:57 Toronto.
- 579f50a — Implement F16-F17 source-cited company research and budgeted career plans. Created November 07 at 17:01 Toronto.
- 0f2b00a — Harden restart-safe drafts, persistence and API setup with desktop verification. Created November 07 at 18:00 Toronto.

All five selected code commits were pushed together on **November 7 at 18:00:29 Toronto** (23:00:29 UTC), advancing main from 12fc14a to 0f2b00a. The GitHub repository Activity API and remote branch SHA confirmed the upload. An additional tracker-history display regression was fixed in 6979590. The free-provider option was added in 8d52ea2 after the user requested it. The five selected commits already satisfy the Period 3 minimum; report-only changes are excluded. Extra commits in other periods do not compensate for a deficient period.

Verification

On November 7, 61 tests passed with zero failures/errors/skips using a desktop-capable Java 21 test runtime and virtual X display. The suite covers PDF/DOCX parsing; preview/confirmation/cancellation; long text, Unicode and control characters; malformed and unsupported model evidence; HTTP contract/configuration precedence and credential redaction; original resume preservation; stale-source rejection; budget checks; exact interview question counts and duplicate detection; session pause/resume/skip/finish and feedback validation; deadline timezone/Observer lifecycle; GUI forms, background responsiveness, actual window and native file-picker cancellation; a separate JVM editing reviewed records; invalid future application dates; and interrupted document editing.

Tests use injected model responses/local HTTP endpoints. They do not establish live Groq/OpenAI account access or prove the truth/quality of every model-generated sentence. Separately, on November

8 the user supplied Windows CLI output reporting: ”Live LLM verification succeeded. The configured model returned the expected JSON response.” The preceding `ai status` output selected Groq and confirmed a locally configured key with its value hidden. PowerShell output confirmed Temurin JDK 21.0.13 and Maven 3.10.0. This user-reported connectivity check was not independently run in the cloud workspace and does not replace feature-level live checks, recordings or human review. Full Stage 3 deterministic/KUMA requirements are outside this Stage 2 checkpoint.

Important corrected defects: long-text JSON regex stack risk; interrupted input retaining stale review state; edited posting descriptions retaining old requirements; unordered collection iteration changing source fingerprints after restart; bracket-containing array values truncating; direct record writes risking partial content; unchecked future application dates; provider error bodies risking disclosure; saved tracker history not appearing in interface output; and native GUI prerequisites missing from the original cloud JRE.

Remaining limitations and finalization work

- Complete each AI workflow against the real model; the user reported that local key setup and ai verify succeeded November 8. Allow time between requests and use short demonstration inputs to fit free-tier quotas.
- Human-review source evidence and qualifications; quotation checks do not guarantee semantic truth.
- Single-candidate local JSON storage; run one interface at a time because there is no cross-process file locking.
- No web browsing/source verification, external application sending or OS/email notifications.
- AI context limit is 60,000 characters. Date reminders use Toronto; interviews use captured original job/rubric context.
- Complete all videos and accessible links, exact AI usage/cost figures and final PDF/human review before submission. Updated diagrams and the five-code-commit Period 3 upload are now present.

7. Demonstration index

Every feature requires an actual running demonstration; screenshots, tests and this report are not substitutes. Use both GUI and CLI across the recordings. Each segment must identify the feature ID, input, output, AI involvement and a relevant error/alternative case. API keys must remain hidden. Video links/timestamps are pending for all 17 features; no fabricated URLs are supplied.

- F01 Candidate Profile Management: profile edit / Skills profile. **Video/timestamp pending.**
- F02 Resume Import: resume import / resume paste / Resumes. **Video/timestamp pending.**
- F03 Resume Skill Extraction: resume extract-skills / Resumes. **Video/timestamp pending.**
- F04 Job Import: job import / Jobs & matching. **Video/timestamp pending.**
- F05 Job Description Analysis: job analyze / job requirements / Jobs & matching. **Video/timestamp pending.**
- F06 Skill Gap Identification: job gaps / job gaps explain. **Video/timestamp pending.**
- F07 Job Matching: job match / job match explain. **Video/timestamp pending.**
- F08 Job Comparison: job compare / Compare jobs. **Video/timestamp pending.**
- F09 Customized Resume Suggestions: resume suggest / Writing assistant. **Video/timestamp pending.**
- F10 Cover Letter Drafting: letter draft / Writing assistant. **Video/timestamp pending.**
- F11 Interview Question Generation: interview questions / Interview preparation. **Video/timestamp pending.**
- F12 Mock Interview: interview start / answer / skip / pause / resume / finish. **Video/timestamp pending.**
- F13 Interview Answer Feedback: interview feedback / Interview preparation. **Video/timestamp pending.**
- F14 Application Tracking: application add / application update / Applications & deadlines. **Video/timestamp pending.**
- F15 Deadline Tracking: deadlines list / deadlines check. **Video/timestamp pending.**

- F16 Company Research Notes: company notes / company show / company summarize. **Video/timestamp pending.**
- F17 Career Development Plan: career plan / Career plan. **Video/timestamp pending.**

Appendix A. Source-file index

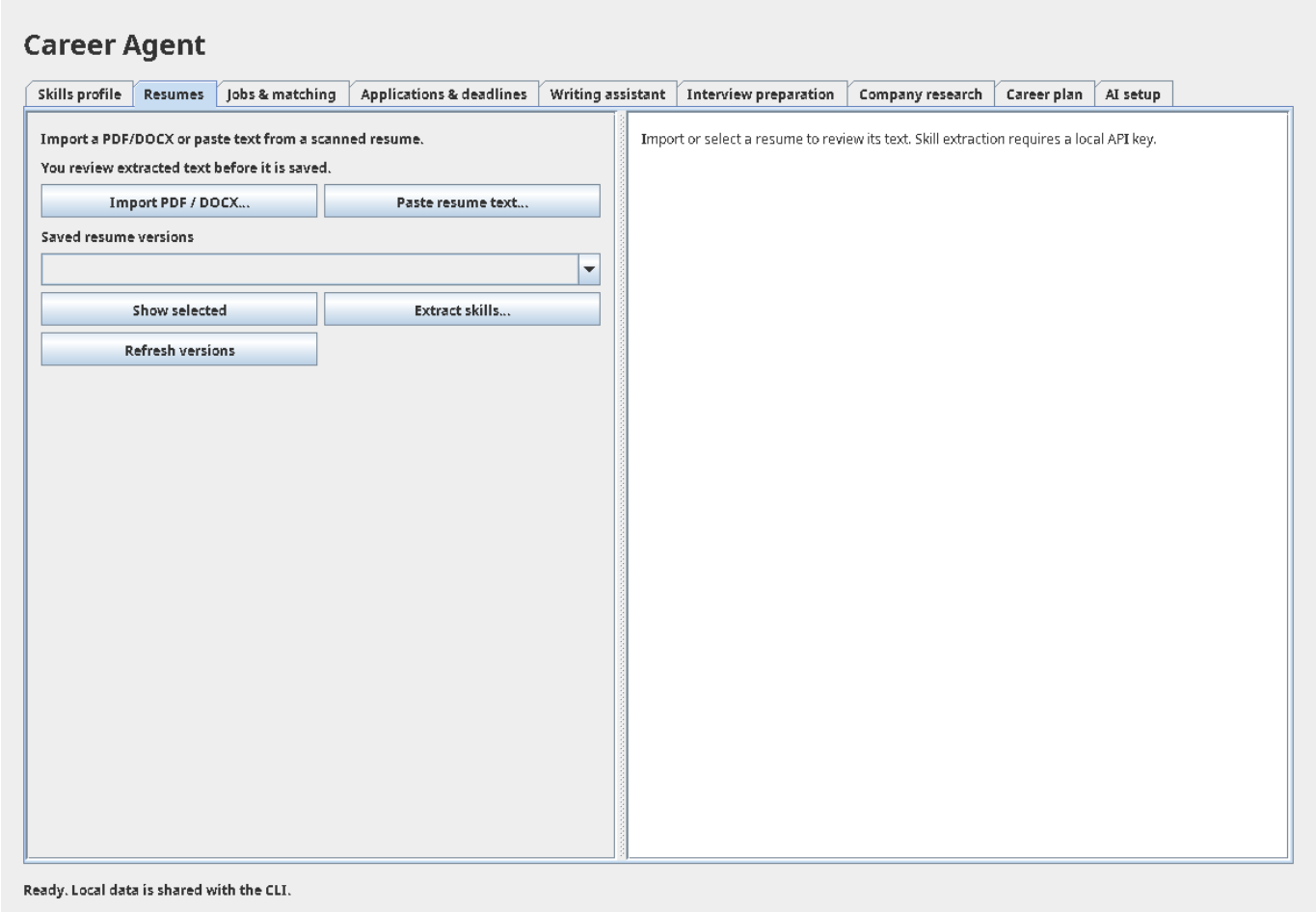
All Java source paths are under `src/main/java/ca/yorku/careeragent/`. Each class referenced above is implemented in its same-named Java file in the package below. This index ties feature/class names to actual files rather than implying unconnected design-only classes.

- `ui/`: `JobSearchCLI.java`, `JobSearchGUI.java`.
- `controller/`: `AgentController.java`.
- `service/`: `CareerAgent.java`, `CareerFacade.java`, `InterviewService.java`, `ToolManager.java`.
- `domain/`: `AnswerFeedback.java`, `Application.java`, `Assessment.java`, `CandidateProfile.java`, `CareerDocument.java`, `CompanyNotes.java`, `InterviewSession.java`, `Job.java`, `JobAnalysis.java`, `QuestionSet.java`, `Requirement.java`, `Resume.java`, `SkillProposal.java`, `SourceFingerprint.java`.
- `persistence/`: `JsonSupport.java`, `LocalRecords.java`, `Repository.java`.
- `parser/`: `DocumentParserAdapter.java`, `ResumeParser.java`.
- `llm/`: `ChatCompletionAdapter.java`, `ConfiguredLLMClient.java`, `GroqAdapter.java`, `Json.java`, `LLMClient.java`, `OpenAIAdapter.java`.
- `strategy/`: `MatchingStrategy.java`, `RequiredSkillMatching.java`, `WeightedMatching.java`.
- `deadline/`: `DeadlineManager.java`, `DeadlineObserver.java`, `DeadlineReminder.java`.

Appendix B. Reproduction and desktop verification

Normal desktop prerequisites: full Java 17+ JDK, Maven 3.9+, local provider API key for model tasks (Groq Free plan by default). Run `mvn package`, then `mvn exec:java` for the CLI or `mvn exec:java -Dexec.mainClass=ca.yorku.careeragent.ui.JobSearchGUI` for Swing. `ai status` reports configuration without printing the key; `ai verify` performs a small actual model call with no personal records, once a key is configured. The API setup section of README explains free Groq configuration and optional OpenAI configuration.

The cloud's original Java runtime is headless-only. Native checks used a user-owned official Adoptium Java 21 desktop JRE and a virtual Xorg dummy display. Standard headless builds skip the one native-window check; desktop-enabled verification runs it. Local/private configuration and data are ignored by Git. Safe fictional sample files are in `examples/`.



No live-key value, private candidate data or invented demonstration links are included in this draft.